Kalshi Market:

Detecting Statistical Arbitrage and Predictive Edges

Bryan Truong

https://github.com/BryanTruongUCLA/PIC16B-Project-.git

Abstract

The goal of this project is to see if there is any edge we can gain on Kalshi to see if we can make more educated predictions on the Tesla markets. The purpose is to get the most accurate model to predict as far into the future as possible, whether a few minutes, or even days. Using the Kalshi API to generate all Kalshi markets relating to large company events, further filtering it down to only large Tesla related markets, allows us to compare to the actual Tesla stock market based on different features.

Table of Contents

Introduction 1-2

Data and Processing 2-4

Methods 4-7

Results 8-16

Conclusion & Future Works 16-17

Bibliography 18

Introduction

Kalshi is a prediction market where users can buy and sell “event contracts” based on the outcome of future events such as political or sport game outcomes. Under the Efficient Market Hypothesis, company-specific outcomes such as Tesla’s stock performance are priced in, meaning that the current stock prices reflect the market's consensus expectation of a future event. With that, Kalshi will be used in comparison to Tesla’s stock market as a reference since Kalshi has so many markets relating to Tesla.

This project explores whether statistical arbitrage opportunities or predictive edges exist within Kalshi’s Tesla-related markets. Specifically, the research aims to answer: Can we use Tesla related Kalshi Markets to predict Tesla price a certain time into the future?

With the market being seen as more of a gamble, we would like to see if we can potentially gain more certainty on the market based on trends. The goal is to potentially find a reliable model to expand to other stock markets and make investing less of a gamble. We hypothesize that Kalshi’s markets have a predictive correlation to future stock market prices which can improve stock market predictions, in our case, Tesla.

All active Kalshi markets referencing publicly traded companies are fetched and validated using multi-stage verification, including API checks, volume checks, and webpage scraping. We gathered six months of minutely candle data for Tesla and S&P 500 and combined it with Kalshi’s Tesla related markets. The model uses Random Forest Classifiers, with Principal Component Analysis for dimensionality reduction to manage the complexity of the feature set.

The report will cover the following sections: Data Acquisition: The process of collecting and preparing the Tesla and Kalshi market data, including feature engineering and data cleaning, Model Architecture: Machine learning models used, including the random forest classifier, dimensionality reduction with PCA, and the methodology, Results and Analysis: Presents the results of the model, including accuracy and confidence analysis, and discusses the performance across different time horizons, Conclusion and Next Steps: Summarize the findings of the project, evaluates the model's effectiveness, and proposes future improvements.

By the end of this project, we aim to demonstrate the viability of using Kalshi's prediction market data to forecast Tesla stock price movements providing insights into financial prediction.

Data and Processing

Data Gathering:

To conduct the analysis, this study utilized the Kalshi Trade API to fetch all **open** markets. We are specifically filtering to 61 markets on Kalshi that are Tesla-related topics. It extracts market prices, volumes, liquidity measures, and timestamps. Our program retrieves all currently open markets, fetches metadata, pricing (yes/no bid/ask), and provides historical candlestick data at 1-minute, hourly, or daily intervals.

Historical minute-level stock data for Tesla (TSLA) and S&P 500 (SPY) was downloaded using the EODHD API, a provider of financial data. The data includes various stock features such as closing prices, opening prices, high/low prices, and trading volumes for each stock. The data spanned a period of 6 months and was provided in 1-minute intervals.

Data Cleaning:

After retrieving the raw data from the two sources, both the Kalshi data and stock data needed to be cleaned and formatted properly before proceeding with analysis. We noticed many gaps in the data set where some minutes were not active. This was resolved through forward-filling where missing values would be filled based on the most recent data point to ensure a consistent timeline. Then closed or paused markets were filled in with zeros to be marked as inactive.

Feature Engineering:

A significant amount of time was actually spent focusing on using useful features from Kalshi and Tesla stock data. Each market on Kalshi had its set of features based on bid and ask prices for the "Yes" option, volume of contracts traded, and open_interest (number of contracts where both sides have the yes side and the no side filled).

- **yes_bid_close** - Current bid price for "Yes"
- **yes_ask_close** - Current ask price for "Yes"
- **volume** - number of contracts traded
- **open_interest** - number of contracts where both sides have the yes side and the no side filled

Velocity features were also implemented in order to understand how prices were changing over time. These were calculated by the price and volume features relative to the time remaining until the market expires.

- **bid_price_velocity** = yes_bid_close / ttm_hours
  - How fast bid price is moving relative to time remaining
- **ask_price_velocity** = yes_ask_close / ttm_hours
  - How fast ask price is moving relative to time remaining
- **price_confidence** = ((yes_bid_close + yes_ask_close) / 2) / ttm_hours
  - How fast the mid-price is changing relative to time remaining
- **interest_velocity** = open_interest / ttm_hours
  - Rate of capital commitment relative to time remaining
- **volume_velocity** = volume / ttm_hours
  - Trading activity relative to time remaining

We then extracted Tesla stock features, as well as the S&P 500 stock features to compare to account for market-wide trends.

Tesla:

- tsla_close
- tsla_open
- tsla_high
- tsla_low
- tsla_volume

S&P 500:

- spy_close
- spy_open
- spy_volume

Our target variable was how the stock market moves, it evaluates whether Tesla's stock price would outperform or underperform the S&P 500 in the future. Based on Tesla's price returns, a classification label is made where the return was greater than 0.5% (UP(1)), less than -0.05% (DOWN(0)), or between -0.05% and 0.05% (NOISE(NaN)).

The final step was to combine the stock data and the Kalshi data to prepare for training. 553 total features were selected after merging the data sets based on the timestamp, where highly correlated features were filtered out to prevent overfitting.

Methods

Random Forest Classifier builds decision trees and makes predictions based on yes/no questions. They model nonlinear decision boundaries well and are robust to outliers making it reliable for our large data set. Since the data contained a lot of features, we implemented PCA from scikit-learn to reduce the dimensionality to prevent overfitting and avoid multicollinearity. Because the Kalshi dataset contains more than 610 engineered features, many of which are correlated, dimensionality reduction is necessary for improving model efficiency and stability. PCA was conducted on the standardized training data and reduced to 10 components, which captured more than 85% of the total variance. It helps provide a better

understanding of the features with the most variance that impact price movement. PCA was exclusively implemented into the Random Forest Classifier because it reduces correlated noise (variation), decreases training time, and lowers overfitting by compressing information. The random forest creates hundreds of decision trees to vote on if Tesla stock price will go up or down, and it will average out the randomness for a stable prediction. This is reliable for our Kalshi market because of the redundant features.

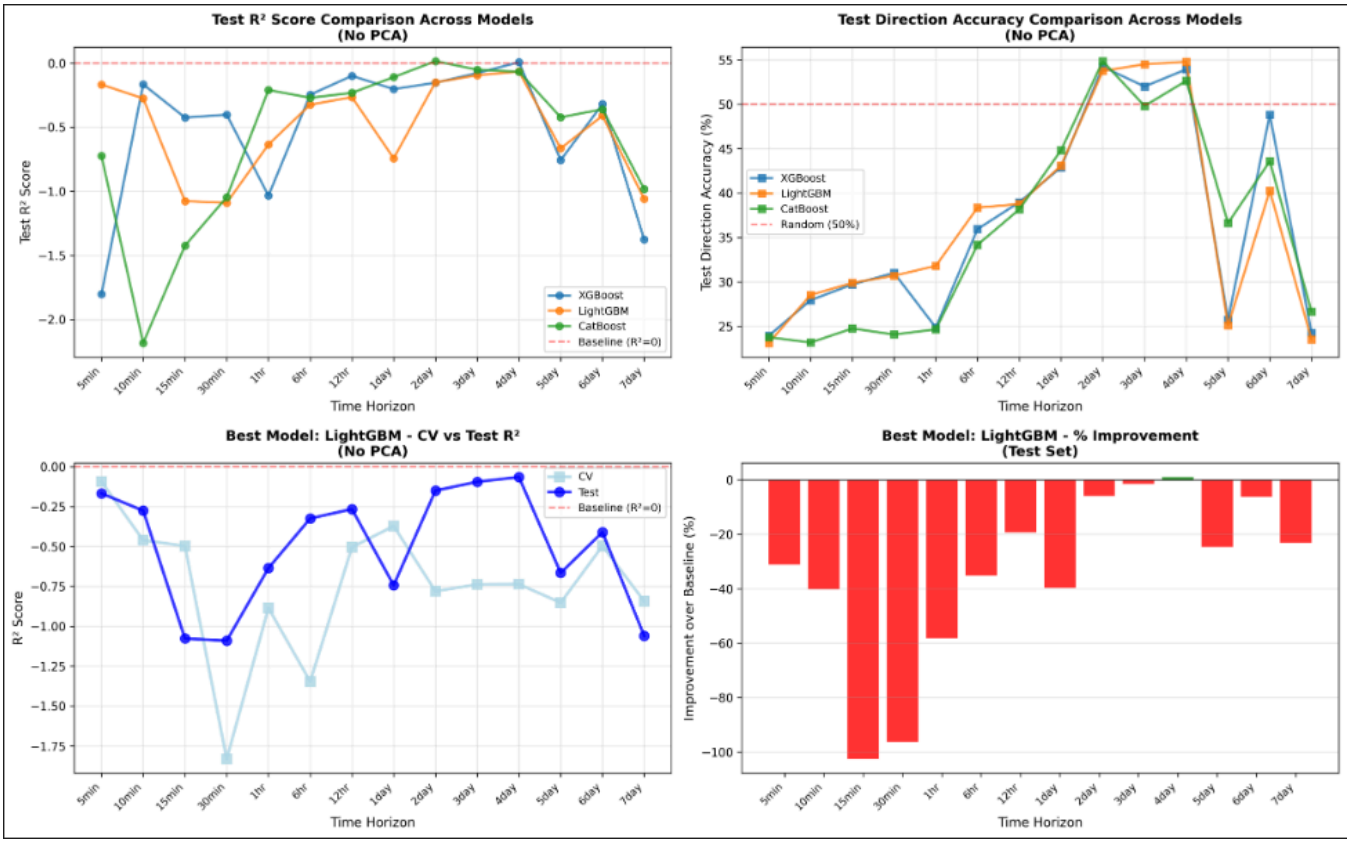


Figure 1: Using a regression model to predict Tesla stock price

Initially our objective was to predict the exact future Tesla price at different forecast horizons but after experimenting, we examined that it was too sensitive to noise and tended to overfit short-term fluctuations and underperform on unseen data. When we used regression to predict price, both noise and signal were being predicted while the majority of the short-term price variance was unpredictable noise

leading to overfitting. Since noise is unpredictable, our R^2 ends up being terrible even if correctly predicting the directional trend (as seen above). For example, if your model correctly forecasts $250 but the stock closes at $241 due to random noise, it penalizes R^2 despite being directionally correct. This creates a situation where gradient boosting models try to fit the noise to improve R^2.

Our Kalshi features provide directional information, not precise information. A regression model cannot output exact prices based on direction. It would be memorizing false correlations in the training data. Mean Squared Error also penalizes large errors, meaning a $10 error is 100 times worse than a $1 error. This makes the model overweigh outliers and create complex interactions that fit training noise rather than generalizable patterns.

Classification solved these problems by collapsing noise into signals. By predicting binary outcomes (up/down), we removed the noise. Trading firms rarely predict exact prices for this reason. Instead they predict direction, magnitude buckets, or probability distributions because the efficient market hypothesis suggests predictable movements are small relative to noise.

Now we introduce the Random Forest Classifier. We produced Confidence Analysis models using three gradient-boosting algorithms: CatBoost, LightGBM, and XGBoost which works best for non-linear data where our features do not influence price direction proportionally. All three are Gradient Boosted Decision Trees, but by comparing them, it would be beneficial in determining the most reliable confidence estimates. These models were not trained with PCA because of its built-in feature selections. Prediction market data contains interactions, for example, a rise in bid price in one market combined with rising volume in another.

Gradient boosting models (XGBoost, LightGBM, CatBoost) are suited for Tesla stock direction prediction with their ability to find complex, non-linear patterns in noisy data. These models work by

building trees sequentially, where each new tree focuses on correcting the errors of previous trees which is crucial because the relationship between Kalshi market sentiment and stock price movements isn't linear.

XGBoost provides the most comprehensive regularization options and handles missing data well, making it robust against overfitting in noisy markets. LightGBM trains significantly faster on large datasets by using leaf-wise tree growth , which focus on the hardest-to-predict samples like days when sentiment contradicts price movements. CatBoost prevents overfitting through "ordered boosting," which ensures the model only learns from past data and never leaks information from the future into predictions. This makes CatBoost particularly stable when market conditions shift.

Unlike neural networks, Gradient boosting models work exceptionally well with a smaller dataset size (thousands vs. millions of samples). They automatically discover feature interactions and provide interpretability so we can understand which Kalshi markets matter most. We will start with LightGBM for fast experimentation, move onto XGBoost for maximum accuracy, and finally CatBoost for its robustness against overfitting and distribution shifts in live market conditions.

Results

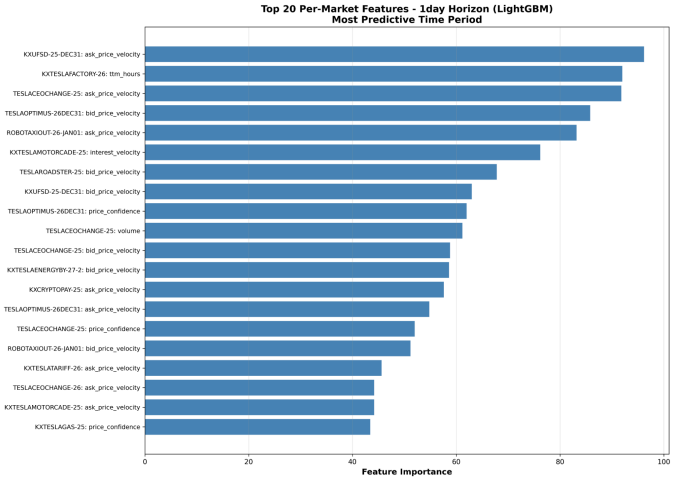


Figure 2: 1 day horizon (LightGBM)

The 1 day horizon LightGBM plot is heavily dominated with the ask_price_velocity and bid_price_velocity across most of the markets.

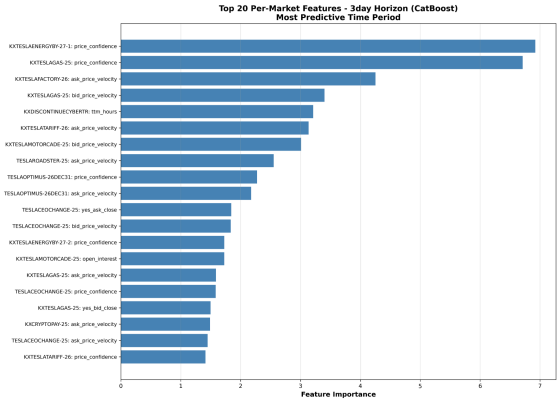


Figure 3:

The 3 day horizon CatBoost is dominated by price_confidence features with Markets such as KXTESLAENENERGYBY and KXTESLAGAS appearing multiple times, with KXTESLAMOTORCADE being the most predictive market.

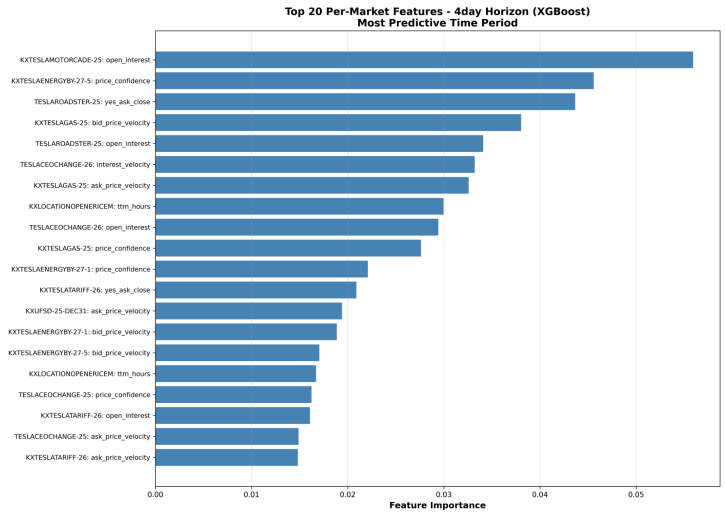
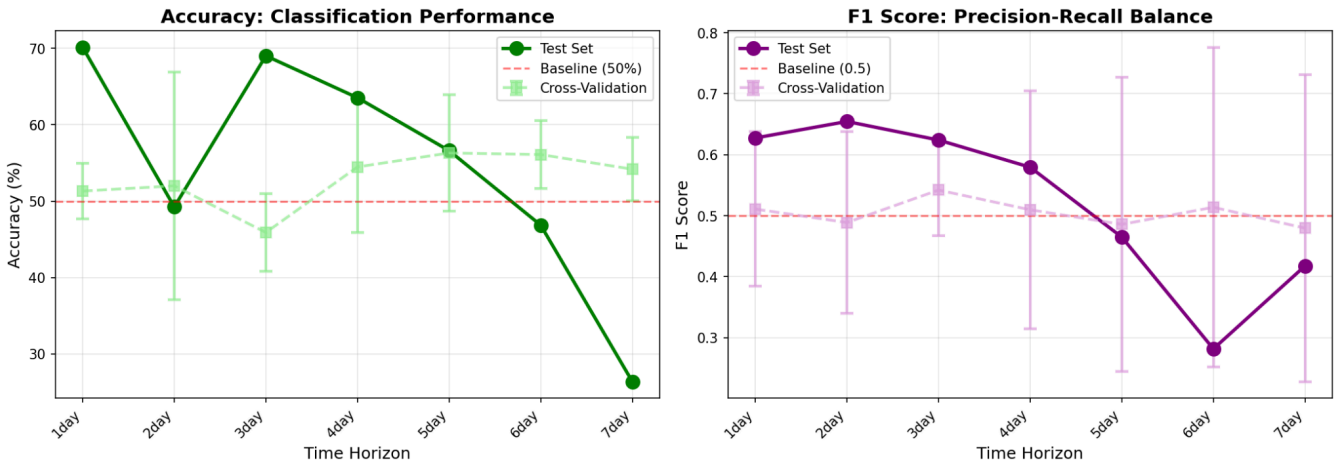


Figure 4:

The 4 day horizon XGBoost's strongest predictor is the open_interest feature for KXTESLAMOTORCADE-25.

In order to assess whether Kalshi's Tesla related markets provide useful information to forecastTesla's stock price, we trained the 3 gradient-boosting models with PCA and created a feature importance chart based on their most predictive time period. We trained LightGBM on a 1-day horizon, CatBoost on a 3-day horizon, and XGBoost on a 4-day horizon to output the top 20 most predictive features per market. A key feature in our models was the time until expiration for each Kalshi Market contract. The reliability of the Kalshi markets increase and become more informative as they approach expiration.

Based on the title of the markets, it would make sense for there to be correlation with the stock market. Most of the features gave a good sense for accurate predictions, but there were still markets that provided misleading information that are related to Tesla but would not affect the stock market, such as "Where will a Tesla diner open this year?"



The performance graphs of the PCA showed poor performance where the accuracy was performing no better than random. The problem with this is that PCA reduces features, completely eliminating important features which made the model run poorly. In order to confirm, ranking the Top 20 per market features will help us track if the features being used are actually important by comparing it to the no-PCA models.

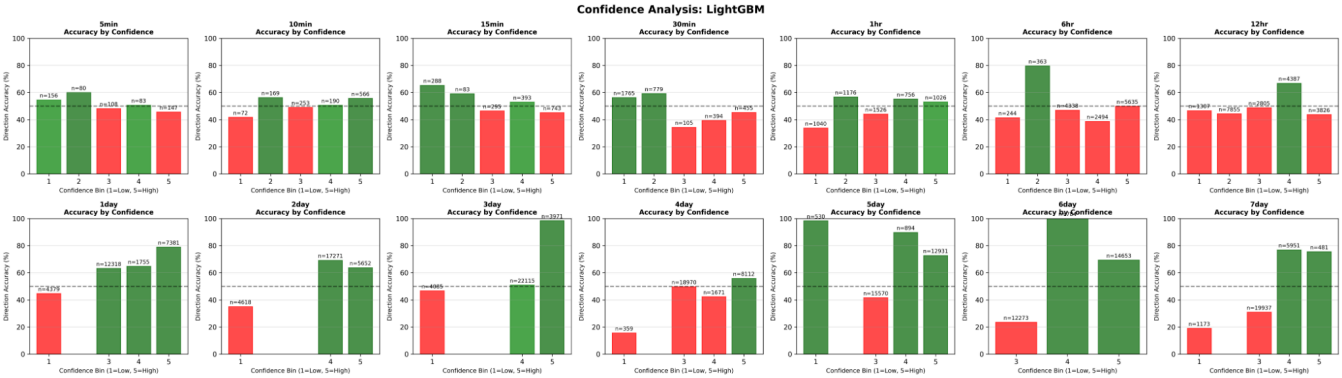


Figure 5

horizon	test_f1
5min	0.3636363636
10min	0.2311557789
15min	0.2896764253
30min	0.1548717949
1hr	0.507090381
6hr	0.5752453373
12hr	0.5226899432
1day	0.6752461872
2day	0.6433509
3day	0.6465491646
4day	0.5023181878
5day	0.556117643
6day	0.4310302434
7day	0.4114530411
	.459

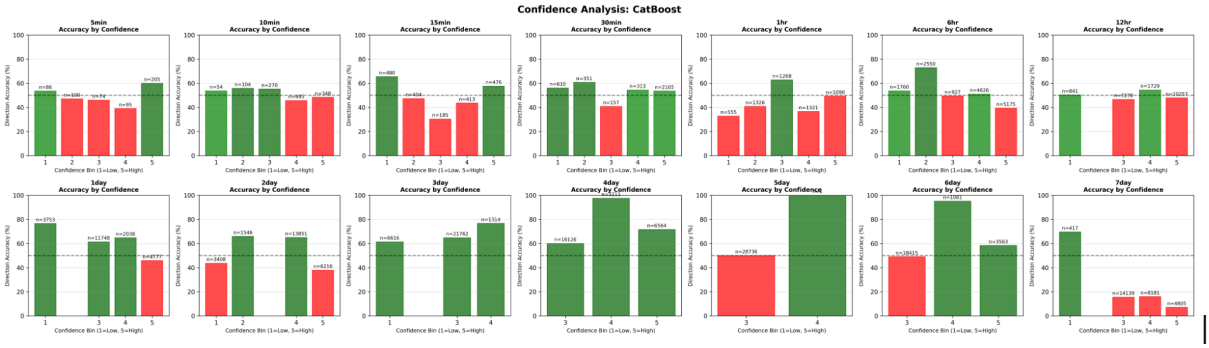


Figure 6

horizon	test_f1
5min	0.5781710914
10min	0.2068965517
15min	0.3216080402
30min	0.1540062435
1hr	0.4719958384
6hr	0.4829422067
12hr	0.5560443279
1day	0.6723249124
2day	0.6205030245
3day	0.6834797057
4day	0.5823297868
5day	0.4251171226
6day	0.2818372627
7day	0.1329927061
	0.441

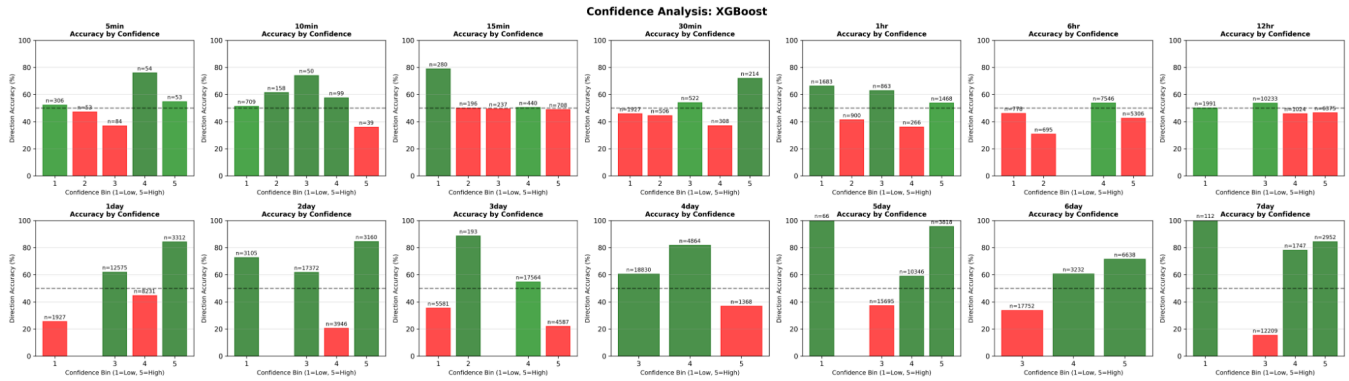


Figure 7

horizon	test_f1
5min	0.5526315789
10min	0.2981651376
15min	0.2829736211
30min	0.2215669756
1hr	0.6028196403
6hr	0.5717795343
12hr	0.5857546073
1day	0.5998087751
2day	0.6331628444
3day	0.6019730601
4day	0.5823297868
5day	0.4409436476
6day	0.2778979759
7day	0.1703036053
	0.465

The confidence analysis evaluates how well the model's prediction confidence aligns with how correct it is. The model outputs a continuous prediction score for each sample, sorts the predictions, and divides them into 5 equal sized bins. Each figure shows 14 bar plots with the first row being the short time periods starting 5 minutes, going up to 12 hours. The second row starts from 1 day and goes all the way up to 7 days. We expect confidence to rise as the Bins increase where Bin 1 indicates the weakest signal, and Bin 5 is the strongest.

Directional accuracy is computed within each bin by comparing the model's predicted direction, up or down, to the true direction for every sample inside the bin. The accuracy is calculated by dividing the number of correct predictions in a bin by the total number of samples in that bin. A prediction is considered correct if the predicted direction matches the actual forecasted movement which will evaluate how well model confidence correlates with correctness and identifies high-confidence, high-accuracy trading opportunities.

CatBoost's directional accuracy for a 3-day horizon performed the best across all bins with a

large sample size. Bin 4 had had a sample size of over 21000 and still performed with high confidence at around 65%. Although the 4-day CatBoost had potentially higher directional accuracy, we found it to be not as reliable due to its smaller sample size. This can be further examined through the F1 scores.

Across all prediction horizons, XGBoost achieved the highest average F1 score of 0.465, indicating that it is the most consistent overall performer. However, the best performing model was the CatBoost 3-day model, which achieved the highest individual F1 score of approximately 0.69. The overall model predicts well with the precision being higher than the benchmark of 0.5. It is able to predict correctly with some mistakes, and while not perfect, would still be profitable. However, there is not enough evidence to say there is an accurate predictability for market forecasting from our models.

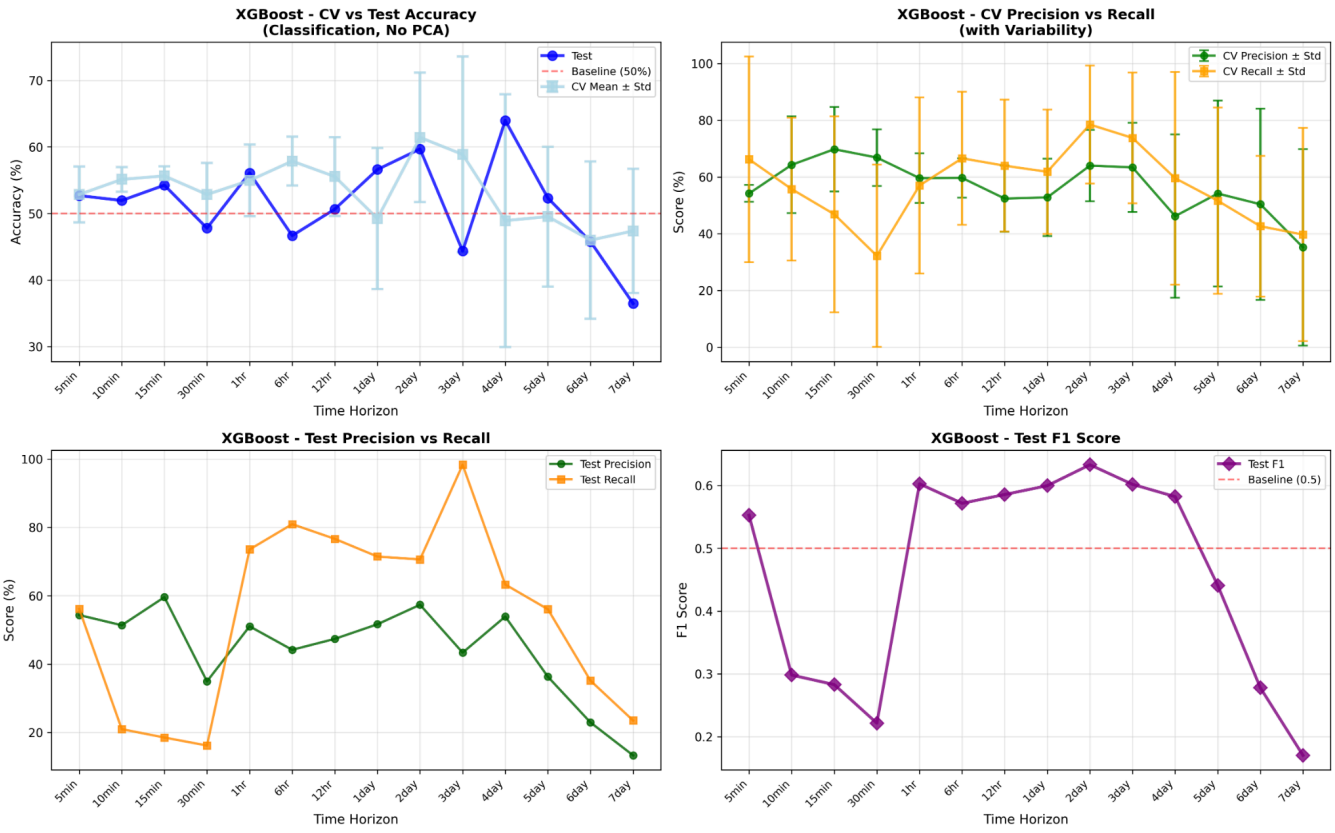


Figure 8

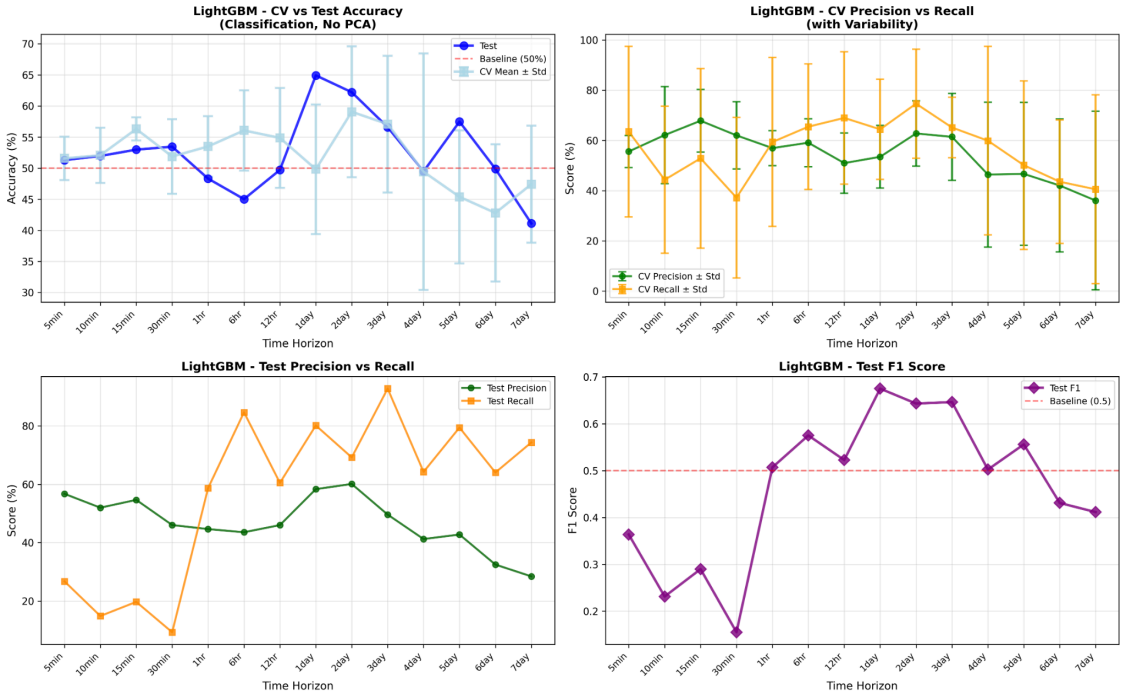


Figure 9

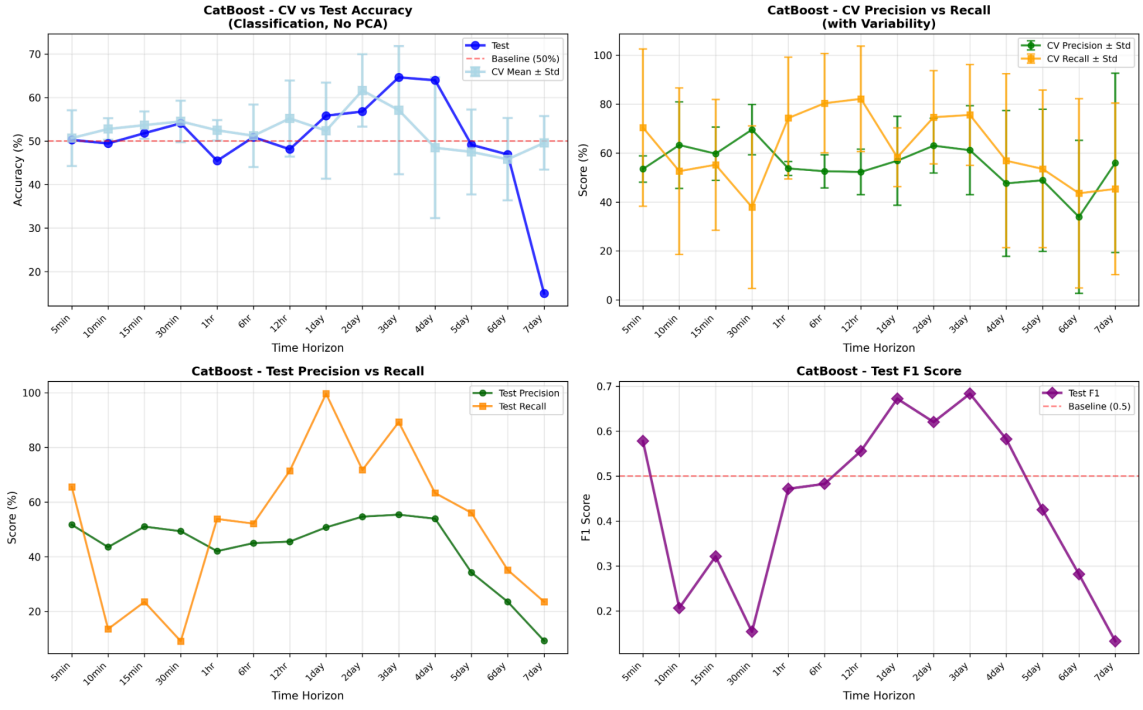


Figure 10

While analyzing three models, XGBoost, LightGBM, and CatBoost, it is clear that short and mid horizon's (up to 3 days) test accuracies and F1 scores occasionally rise above the 50% baseline, but the improvements are small and inconsistent. Precision and recall vary across horizons, and none of the models produce stable, reliable performance at longer horizons where accuracy, recall, and F1 collapse. While there are certain peaks (1-3 days), these spikes are not consistent and differ between models, making it more likely to be due to noise than predictive signal.

Across the time horizons, the cross-validation standard deviations show that the models show there is a lot of variation within each fold, indicating that it is very inconsistent across folds. In the early horizons (5–30 minutes), CV accuracy, precision, and recall tend to have manageable levels of spreads, showing some instability but still staying within relatively manageable bounds. When the horizon extends (1–3 days), the error bars widen a lot, with some CV precision and recall intervals being over 40–60 percentage points. This widening suggests that model performance becomes highly sensitive to data splitting, meaning that certain folds contain patterns that are either especially favorable or unfavorable for learning. The large standard deviations imply reduced generalizability and highlight that mean CV scores alone may overstate reliability.

Conclusion & Future Works

The goal of this project was to determine whether Kalshi's Tesla-related markets contain predictive information about future Tesla stock price movements. Using six months of minute-level market and price data, combined with features from Kalshi's event contracts, we trained several machine learning models including Random Forests with PCA, and gradient-boosting classifiers such as XGBoost, LightGBM, and CatBoost across multiple forecast horizons ranging from minutes to several days.

Overall, the results show limited predictive power. While certain individual models, particularly the CatBoost 3-day horizon model, achieved relatively strong performance, across all horizons shows that most models perform no better than random guessing. The highest average F1 score across horizons was 0.465 from XGBoost, which is lower than the baseline.

These results indicate that, although some Kalshi markets appear correlated with future Tesla movements on specific days, the aggregate signal is weak, noisy, and unstable. Many features that appeared predictive in one horizon lacked consistency in others, suggesting that temporary patterns between Kalshi and the Tesla market may have created short-lived edges that do not generalize well. The directional accuracy analysis further revealed that some high-confidence predictions were based on relatively small sample sizes that may reflect noise rather than true predictive structure. Our results were effectively random, with only occasional predictability.

For future improvements to the model's predictive capabilities, significantly expand the feature space and depth of analysis applied to Kalshi market data. This will be done using richer order book based signals, such as liquidity imbalances, multi-level depth metrics, order-flow intensity, and probability-curve shape features. We also want to get higher data quality and better data sources by integrating external datasets such as news sentiment, macroeconomic indicators, and sector-relevant prediction markets.

We also want to improve the modeling pipeline by incorporating neural network architectures such as convolutional networks and transformer-based models to better capture Kalshi signals and TSLA price action. I believe these improvements should allow much better pattern identification and higher accuracy.

Bibliography

Kalshi API. Kalshi Technologies Inc. Kalshi API Documentation. Retrieved from https://trading-api.readme.io/reference/getting-started. Accessed 2024. Used to retrieve prediction market sentiment data for Tesla-related contracts.

EODHD (End of Day Historical Data). EODHD Financial APIs. Retrieved from https://eodhistoricaldata.com/. Accessed 2024. Used to obtain historical stock price data, trading volumes, and technical indicators for Tesla Inc. (TSLA).

OpenAI API. OpenAI. ChatGPT API Documentation. Retrieved from https://platform.openai.com/docs/. Accessed 2024. Initially explored for sentiment analysis but found to be unreliable for our use case.

López de Prado, M. (2018). Advances in Financial Machine Learning. John Wiley & Sons. Referenced for time-series cross-validation methodology and feature engineering techniques specific to financial data.